

INVARUNE BY NIMESHBUILD

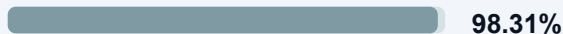
Broader inspection. Measured against the same evidence.

Benchmark update / Invarune 0.14.0

The unchanged development-visible fixture corpus and eight pinned public projects were scanned again after targeted detector fixes. This publication records the measured changes, comparison scope and unresolved work.

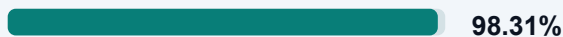
Precision

v0.13.0



98.31%

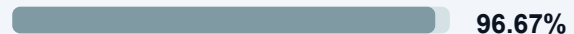
v0.14.0



98.31%

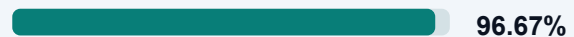
Recall

v0.13.0



96.67%

v0.14.0



96.67%

Unchanged fixture comparison	Observed result
Cases / original corpus	113 / 1.2.0
Previously failing cases now correct	0
New mismatches on previously correct cases	0
Remaining labeled mismatches	3
Pinned public-project scope	8 projects / 4,120 exported files
Current Invarune source observations / coverage gaps	146 / 21

These percentages describe project-authored rule-presence labels used during development. They are not production accuracy, independent validation or a claim that Invarune is better than every scanner.

[Interactive benchmark dashboard and downloadable evidence](#)

Read the evidence in layers

Fixture accuracy and unchanged labels

What was fixed and what still needs work

Source scope and release additions

Fresh comparison on eight public projects

Finding overlap and complementary coverage

Why use invscan in an agent/MCP review

Reproduce, validate and inspect the complete ledger

Evidence layer	What it establishes
Deterministic fixtures	Agreement with explicitly authored rule-presence labels; supports regression and before/after analysis.
Public source observations	Patterns reported by configured tools on exact pinned files; requires source and deployment review.
Optional model review	Advisory judgments only when actually requested and completed; not a replacement for independent truth.
Runtime validation	Still needed for authentication, reachability, tenant isolation, prompt-injection resistance and deployed safeguards.

All measurements in this update preserve target isolation: no upstream project code, service or exploit was executed. Tool support, parser warnings and incomplete work remain visible.

Fixture accuracy

The before and after runs use the exact same 113 cases, source bytes, paths and expected labels. Corpus SHA-256:

eb7f1eba93f8e9842634cbd12687dde5bea879505d99d367621214583e54dde3

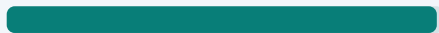
Precision

v0.13.0



98.31%

v0.14.0



98.31%

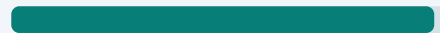
Recall

v0.13.0



96.67%

v0.14.0



96.67%

Outcome	Before 0.13.0	After 0.14.0
True positive	58	58
True negative	52	52
False positive	1	1
False negative	2	2
Analysis errors	0	0

Precision = $TP / (TP + FP)$. Recall = $TP / (TP + FN)$. A true-positive label here means that the designated detector correctly recognized the fixture's specified source pattern. It is not an independently confirmed vulnerability.

All challenge failures remain in these denominators. Default regression gating is a narrower CI policy and must not be read as zero mistakes. Fixes were developed with knowledge of these cases; this is a transparent development benchmark, not a held-out test.

[Before accuracy JSON](#)

[After accuracy JSON](#)

Fixes and remaining limits

No prior fixture outcome changed in this release

The unchanged baseline cases retain their prior outcomes. The separate skill/tool measurements on the next page describe the added inspection scope; they do not increase the original corpus denominator.

Still unresolved on this corpus

Fixture	Remaining mismatch
gap-interprocedural-url	Cross-function argument flow needs interprocedural analysis.
gap-js-wrapper-taint	JS wrapper argument flow is not whole-program taint analysis.
gap-validated-user-url	Constraint solving could prove this branch restricts the request to a fixed destination.

The release's paired and adversarial regression tests cover the supported interpretations and counterexamples. General interprocedural flow, dynamic runtime values, full YAML semantics and arbitrary shell expansion are not proven by these targeted changes. Review an uncertain finding in context instead of silently suppressing it.

[Detailed change analysis and remaining gaps](#)

Skills, tools and honest scope

On the unchanged 4,120 exported files, Invarune observations changed from 146 to 146. The final scan retains 21 analysis gaps. New metadata inspection can expose additional unknowns without changing the finding count.

Four new deterministic checks

Rule	Risk evidence
AI043	Explicit instruction-hierarchy overrides in supported skill/tool text.
AI044	Sensitive-data transfer instructions with an explicit destination.
AI045	Concealed actions and approval/security-boundary bypass directives.
AI046	Literal read-only annotations contradicting destructive tool descriptions.

Separate measurement: 81 cases / 331 rule-presence assertions. TP 40; TN 288; FP 0; FN 3. Precision 100.00%; recall 93.02%. All authored challenge misses remain in these denominators.

The separate skill/tool corpus is not added to the original 113 assertions. Each label describes a source pattern, not malicious authorship or exploitation. The public-source export excludes most skill documentation and cannot measure complete skill-package coverage.

[Separate skill/tool measurement and known misses](#)

[Every scan: algorithms, controls, sources and uncertainty](#)

Interpreting a score

There is no overall security percentage. Findings use exact severity counts. Partial mapping reach = active selected controls with an active mapped rule / active selected controls. Optional AI answer coverage = uniquely answered active checks / active selected checks. Both are multiplied by 100; zero denominators are null. Neither is a control pass rate.

Exceptions receive no pass credit. AI may add advisory concerns or help investigate inconclusive evidence; it cannot clear static findings or coverage gaps. Missing runtime proof remains unresolved.

[Exact source observation delta](#)

Eight projects. Fresh executions.

The same exported first-party source/configuration bytes were offered to Invarune, Semgrep CE, Bandit and Gitleaks. Equal input does not make their languages, rule scope or supported inputs identical. Counts below are observations, not unique vulnerabilities.

Project	Files	Invarune	Semgrep	Bandit	Gitleaks
autogen	786	11	6	183	0
crewai	968	27	8	232	0
fastmcp	283	0	2	95	1
github-mcp	163	15	1	unsupported	0
langgraph	211	23	8	84	1
mcp-reference	86	61	0	0	0
openhands	1237	9	0	unsupported	1
pydantic-ai	386	0	2	344	0

Tool	Recorded version	Observations	Scope status
Invarune	0.14.0	146	3 completed; 5 completed_with_analysis_gaps
Semgrep CE	1.177.0	27	1 completed_no_findings; 3 completed_with_analysis_errors; 4 completed_with_findings
Bandit	1.9.4	938	1 completed_no_findings; 1 completed_with_analysis_errors; 4 completed_with_findings; 2 unsupported_no_python_files
Gitleaks	8.30.1	3	5 completed_no_findings; 3 completed_with_findings

Invarune reports 21 coverage gaps. Those inputs are incomplete, not clean. Semgrep parser warnings, Bandit parser errors and unsupported exports are retained in the run-status receipts. All eight Invarune projects have two byte-identical repeated outputs under the recorded runtime and settings.

[All normalized run statuses and coverage details](#)

What overlaps - and what does not

A compatible family, the same pinned project/path and intersecting line spans are required for a match. One-to-one pairs require exactly one candidate on each side. Ambiguous and location-only relations remain separate; matching does not establish a true-positive label.

Compared with	1:1 pairs	Ambiguous edges	Invarune unmatched	Other tool unmatched
Semgrep CE	7	0	139	20
Bandit	12	5	129	925
Gitleaks	1	0	145	2

The complete ledger exposes what each scanner reported, the other-tool match candidates, exact pinned paths and rule-family predicates. A tool-only record may reflect useful additional scope, a differently defined rule, unsupported analysis, a false alarm or a genuine missed risk; count alone does not decide which.

Native findings remain review signals

Broad source scanners can surface assertions, weak-hash uses, serialization APIs and import-only records. Invarune emphasizes selected agent/MCP configuration and execution risks with control/source mappings. Secret scanners, dependency/CVE scanners and runtime/adversarial tests add other evidence. No tool is awarded false positives merely because another tool lacks the same rule.

No new ground-truth percentage is claimed for public projects

This deterministic rerun does not perform Claude adjudication or independent exploit validation. Earlier model-authentication failures remain historical failures, and earlier selected source audits remain separate evidence. Confirmed-vulnerability precision and public-project recall are unavailable.

[Complete finding-by-finding ledger](#)

[Machine-readable overlap and unmatched sets](#)

An agent/MCP review workflow

Invarune's value is the traceable review workflow around selected security checks. These are product capabilities documented and tested by this project, not a score against untested competitor editions.

Capability	How to use the evidence
Source or built Linux image	Inspect supported source/configuration and image metadata without starting target code or a container. Compiled-only logic remains unassessed.
Offline control explorer	Use <code>invscan --ask</code> and explain commands without a model, login or network. Each answer explains control meaning, sources and limits.
46 rules / 66 controls / 132 checks	Rules partially map to 30 controls; the remaining controls and complete acceptance checks need other evidence. No inferred compliance pass.
Concrete remediation	Reports connect findings to agent/MCP context, proposed fixes, verification steps and possible mitigating layers. Undeployed defenses do not lower risk automatically.
Optional bounded AI review	Enable official CLI or API/gateway review explicitly. Deterministic validation constrains evidence/citations/budgets; model judgments stay advisory.
Review and justification roundtrip	Carry edited supported report inputs into a fresh scan. Accepted justifications stay justified, excluded from the applicable scored denominator, and retain evidence bindings.

Use complementary runtime validation for MCP authorization, tenant separation, sandbox escape resistance, egress enforcement and prompt-injection task outcomes. A repository or image cannot establish the deployed identity, network policy or human approval process.

[CLI quick start](#)

[Controls, benchmarks and primary source mappings](#)

Reproduce and audit the result

Every published count is derived from the linked local execution artifacts. The PDF builder rejects mismatched fixture hashes/labels, invalid ledger totals, overlap partition errors, mismatched scanner versions and changed before/after project inputs.

Input	SHA-256
accuracy-before.json	49bd49b0464b288beca29212d1b8be4f3b6c48ce5037a0da771bc2c829de9e38
accuracy-after.json	4d8c0f5213eedb3879986665757e7f5e17689dd26417642df7fa8b2d62874a27
observations.json	f4665347052d4d0aa5a91ced89c755f3b9d89e0f7271db59eda8c1ad3b7a3355
overlaps.json	7c9e5af99024852b3a7ca317cb1a5ffb316f5b5a413d9f056bf9719dde98c6d8
run-status.json	f188c56ddcc14061deaf8c920bfd0758d327b96d28d784cb4d94d8caaed8a93d
baseline-identity.json	ee1fb67aff3f3a98af54c08fa3dd9736158ac2fcea658b0def15da83144ed542
before-after-accuracy.json	75d97f72c4bc931904cc50b6670fc7d7162dcd199477a33efb5844723675b5a4
skills-tools-accuracy.json	90a10ab28ec2af73bf31529c30bda830a3aa06bb1ce789cead212a6908355b8d
skills-tools-evaluation-receipt.json	a71bd9d30543381aab32e61ef692b284e4a6f232c50d980a3725ab0075f3764d

Reproduction sequence

1. Read this benchmark README and exact tool/source lock.
2. Recreate the pinned source exports and verify their manifests.
3. Run the frozen baseline and final scanner independently; preserve both receipts.
4. Run the recorded external versions and rule pack on the same exports.
5. Regenerate the exhaustive observation/overlap ledger and fixture evaluations.
6. Build the dashboard/PDF from those receipts, then render and visually inspect every page.

[Reproduction protocol and tool configuration](#)

[Source selection and pinned repositories](#)

[PDF and site authoring instructions](#)

Invarune by NimeshBuild is an original engineering synthesis. Referenced publishers and compared projects do not endorse it. Third-party materials retain their own terms. Public repository visibility does not independently grant an open-source license.